# 🚀 NİHAİ TEKNİK ÜRÜN GEREKSİNİM DOKÜMANI (PRD)

## Proje: NexusOSINT (MVP v1.0) - CPO & Architecture Deep Dive

**Tarih:** 15 Nisan 2026

**Gizlilik Seviyesi:** C-Level & Mühendislik Ekibi Özel

**Sürüm:** v1.0.0-RC1

## 1. MİMARİ VE TEKNOLOJİ YIĞINI (TECH STACK DEEP DIVE)

Geliştirme ekiplerinin kullanacağı spesifik versiyonlar ve altyapı standartları:

### 1.1. Frontend (İstemci Tarafı)

- **Core Framework:** Next.js 14 (App Router kullanılarak SSR/SSG optimizasyonu).
- **Styling:** Tailwind CSS (Global konfigürasyon, CSS variables ile Dark/Light mode desteği zorunlu).
- **State Management:** Zustand (Sadece global state için: User Auth objesi, Kredi bakiyesi, Aktif Seçili Düğüm. Component state'leri için React `useState` kullanılacak).
- **Data Fetching:** React Query (TanStack Query v5) (Sık güncellenen Delta View verilerini cache'lemek ve stale-time yönetimi için).
- **Grafik Motoru:** D3.js (v7) + React-Force-Graph (Canvas render ile performans optimizasyonu. DOM node'ları yerine Canvas kullanılarak 500+ düğümde kasma engellenecek).

### 1.2. Backend & NLP (Sunucu Tarafı)

- **Core Framework:** FastAPI (Python 3.11+). Tüm I/O işlemleri (veritabanı, dış API çağrıları) `async/await` mimarisinde yazılacak.

- **NLP/NER Engine:** SpaCy (v3) üzerine fine-tune edilmiş, regex (e-posta, cüzdan adresi, API key) ve entropi tabanlı (şifre hashleri) karma bir varlık çıkarma pipelını.
- **Task Broker & Worker:** Celery (v5) + Redis (v7). Arka plan taramaları için zorunlu.

### 1.3. Veri Katmanı (Data Layer)

- **Primary DB:** PostgreSQL 16. (Supabase üzerinden managed).
- **ORM:** SQLAlchemy 2.0 (async modda) veya Prisma.

## 2. VERİTABANI ŞEMASI (DATA DICTIONARY)

MVP için gerekli olan temel tablolar ve kısıtlamalar (Constraints).

### Table: `targets` (İzlenen Hedefler)

| Kolon Adı | Veri Tipi | Kısıtlamalar (Constraints) | Açıklama |
|---|---|---|---|
| `id` | UUID | Primary Key, Auto-gen | Hedefin tekil ID'si |
| `user_id` | UUID | Foreign Key -> `users.id` | Hedefi oluşturan kullanıcı |
| `target_value` | VARCHAR(255) | Not Null, B-Tree Index | Örn: "hedef.com" veya "isim@hedef.com" |
| `is_monitored` | BOOLEAN | Default: False | "Sürekli İzle" (Daily Delta) aktif mi? |
| `last_scanned_at` | TIMESTAMPTZ | Nullable | Son taramanın bitiş zamanı |

### Table: `entities` (Çıkarılan Varlıklar/Düğümler)

| Kolon Adı | Veri Tipi | Kısıtlamalar (Constraints) | Açıklama |
|---|---|---|---|
| `id` | UUID | Primary Key, Auto-gen | Varlığın tekil ID'si |
| `target_id` | UUID | Foreign Key -> `targets.id` | Hangi hedefe ait olduğu |
| `entity_type` | ENUM | Not Null | EMAIL, PWD_HASH, API_KEY, SUBDOMAIN, COMMIT |

| | | | |
|---|---|---|---|
| `raw_value` | VARCHAR | Not Null | Maskelenmemiş orijinal veri (Şifreli tutulmalı) |
| `confidence` | FLOAT | 0.0 ile 1.0 arası | NLP/Regex modelinin güven skoru |
| `source_url` | TEXT | Not Null | Verinin çekildiği GitHub/Pastebin linki |
| `is_ignored` | BOOLEAN | Default: False | Kullanıcı False-Positive olarak işaretledi mi? |
| `discovered_at` | TIMESTAMPTZ | Default: NOW() | Daily Delta mantığı bu kolona göre çalışır |

# 3. CORE API SPESİFİKASYONLARI (REST & WEBSOCKET)

## Endpoint 1: Tarama Başlatma (Asenkron)

- **Route:** `POST /api/v1/scans`
- **Auth:** Bearer Token (JWT) zorunlu.
- **Request Body:**

JSON
```
{
  "target": "example.com",
  "scan_type": "full_osint"
}
```

- **Response (202 Accepted):** (İşlem uzun süreceği için direkt veri dönmez, Job ID döner).

JSON
```
{
  "job_id": "req_8f72c1a...",
  "status": "processing",
  "message": "Scan queued successfully. 1 credit deducted."
}
```

- **Error Responses:**
  - `402 Payment Required`: Kullanıcının kredisi 0 ise.

- `429 Too Many Requests`: Kullanıcı son 1 dakika içinde 5'ten fazla tarama başlattıysa.

### Endpoint 2: Grafik Verisini Çekme

- **Route:** `GET /api/v1/targets/{target_id}/graph?timeframe=24h`
- **Response (200 OK):** (D3.js'in beklediği formatta `nodes` ve `links` array'leri).

JSON
```
{
  "nodes": [
    {"id": "e_1", "group": "EMAIL", "label": "admin@ex.com",
"is_new": false},
    {"id": "e_2", "group": "API_KEY", "label": "AKIA*****",
"is_new": true}
  ],
  "links": [
    {"source": "e_1", "target": "e_2", "relationship":
"authored_commit"}
  ]
}
```

## 4. BDD HİKAYELERİ VE EDGE CASE (İSTİSNAİ DURUM) YÖNETİMİ

Standart "mutlu yol" (happy path) dışında, sistemin kırılacağı noktaların spesifikasyonları.

### Epic 1: Veri Toplama Motoru (Ingestion Engine)

#### Story 1.1: GitHub API Rate Limit (Sınır) Aşımı Durumu

- **Given:** Celery worker, bir hedefin GitHub commitlerini çekerken.
- **When:** GitHub API'sinden `403 Forbidden (Rate Limit Exceeded)` yanıtı dönerse.
- **Then:** Sistem işlemi "Failed" durumuna geçirmemeli.
- **And:** Exponential Backoff (Kademeli Gecikme) algoritması ile task'i 15 dakika sonrasına yeniden kuyruğa (retry queue) almalı.
- **And:** Kullanıcının arayüzündeki tarama durumu "Geçici olarak duraklatıldı, API limiti bekleniyor" olarak güncellenmeli.

**Story 1.2: Dorking - SerpApi Yanıt Vermeme (Timeout) Durumu**

- **Given:** SerpApi üzerinden Google Dorking sorgusu atıldığında.
- **When:** 30 saniye içinde sunucudan yanıt alınamazsa (Timeout).
- **Then:** Celery task'ı bu adımı "Skipped" (Atlandı) olarak işaretlemeli.
- **And:** Kullanıcıya "Arama motoru taraması zaman aşımına uğradı, kısmi sonuçlar gösteriliyor" şeklinde uyarı (Warning Toast) çıkarılmalı. Kesinlikle tüm tarama iptal edilmemelidir.

## Epic 2: Veri İşleme ve Hook Bildirimleri

**Story 2.1: Hassas Verilerin Maskelenmesi (GDPR/KVKK Uyumu)**

- **Given:** Crawler içinde "AKIA1234567890ABCDEF" şeklinde bir AWS API anahtarı bulduğunda.
- **When:** Veri PostgreSQL'e kaydedilirken ve Frontend'e basılırken.
- **Then:** Veritabanında `raw_value` AES-256 algoritmasıyla şifrelenerek saklanmalı.
- **And:** Frontend API response'unda (JSON) veri kesinlikle `AKIA******CDEF` şeklinde maskelenmiş olarak iletilmeli. Tarayıcı Network tabından orijinal string görünmemelidir.

## Epic 3: Arayüz (UI) State (Durum) Matrisi

Geliştiriciler aşağıdaki UI durumlarının her biri için tasarım (Figma) karşılığını kodlamak zorundadır:

| Bileşen | Empty State (Boş) | Loading State (Yükleniyor) | Error State (Hata) |
|---|---|---|---|
| **Dashboard** | Ortada "Henüz tarama yapmadınız, ilk hedefinizi ekleyin" illüstrasyonu. | Skeleton loaders (gri, kayan barlar). | "Veri çekilemedi. Yeniden Dene" butonu. |
| **Knowledge Graph** | (N/A - Graph sadece veri varken açılır) | Ortada dönen radar animasyonu. | Graph alanında "Bağlantı koptu" uyarısı. |
| **Context Panel** | Sağ panel gizlidir. | Düğüme tıklandığında veri gelene kadar spinner. | "Kaynak URL bulunamadı." metni. |

# 5. İŞ MANTIĞI VE FİNANSAL ALGORİTMALAR (BUSINESS LOGIC)

### Kredi Düşüm ve Cüzdan Algoritması

1. **Race Condition (Yarış Durumu) Önlemi:** Bir kullanıcı aynı anda 5 farklı sekmeden "Tara" butonuna basarsa, bakiye eksiye düşmemelidir.
    a. *Teknik Zorunluluk:* Cüzdan güncelleme işlemi veritabanında **Row-level Lock** (`SELECT ... FOR UPDATE`) ile veya Redis atomic decrement (DECR) işlemi ile yapılmalıdır.
2. **Daily Delta Ücretlendirmesi:** "İzlemeye Al" işlemi günlük 0.5 kredi düşer.
    a. *Senaryo:* Kullanıcının 0.2 kredisi kaldıysa.
    b. *Aksiyon:* Gece 00:00'da çalışan Cron job, bakiyenin yetersiz olduğunu tespit eder. Tarama **yapılmaz**. Kullanıcıya "Bakiyeniz yetersiz olduğu için X hedefinin izlenmesi durduruldu" e-postası (Tetikleyici/Trigger) atılır. (Bu, harika bir dönüşüm (conversion) tetikleyicisidir).

# 6. TELEMETRİ VE ÜRÜN ANALİTİĞİ MATRİSİ (POST-LAUNCH)

Hangi butonun işe yarayıp yaramadığını anlamak için frontend'e gömülecek spesifik Event'ler (Örn: Mixpanel veya PostHog kullanılacak):

- **Event Name:** `scan_initiated`
    - *Properties:* `target_type` (domain/email), `user_credit_balance_before`
- **Event Name:** `graph_node_ignored`
    - *Properties:* `entity_type` (örn: EMAIL), `confidence_score` (Algoritmanın nerede hata yaptığını anlamak için çok kritik).
- **Event Name:** `export_downloaded`
    - *Properties:* `format` (PDF/MD), `node_count` (İnsanlar harita ne kadar büyüdüğünde export alma ihtiyacı duyuyor?).

# 7. KABUL KRİTERLERİ (DEFINITION OF DONE - DoD)

Bir Jira ticket'ının (görevin) QA ekibi tarafından "Bitti" (Done) olarak kabul edilmesi için:

1. Birim testleri (Unit Tests) yazılmış ve test coverage (kapsama) oranı en az %80 olmalıdır (Pytest).
2. Next.js bundle boyutu uyarısı vermemelidir (İlk yükleme 150kb'ı geçmemeli).
3. Uçtan uca (E2E) testler Cypress veya Playwright ile yazılmış ve CI/CD pipeline'ından (GitHub Actions) yeşil ışık almış olmalıdır.
4. D3.js haritası, 500 düğüm varken pan/zoom işlemlerinde 60 FPS (Frame Per Second) altına düşmemelidir. Altına düşerse kod refactor edilecektir.